

LABERINTO DE RUTAS

Algoritmos de búsqueda · RA1

Programación de Inteligencia Artificial

Alumno: Aimar Duarte

Curso 2026

1. Objetivo del proyecto

El objetivo de este trabajo ha sido crear una aplicación web con un tablero de 20×20 celdas para probar y comparar distintos algoritmos de búsqueda. En el tablero se puede marcar un punto de inicio, una o varias metas, obstáculos y celdas con un coste mayor.

Los algoritmos principales que he implementado son BFS, DFS, búsqueda de coste uniforme (UCS) y A*. Además, añadí una búsqueda bidireccional como mejora. La aplicación muestra visualmente cómo se explora el mapa y al final enseña los pasos, el coste y el número de celdas exploradas.

La idea no era hacer una copia exacta del ejemplo de clase, sino conseguir un funcionamiento parecido y que sirviera para entender las diferencias entre los algoritmos.

2. Paso a paso de construcción

Empecé por la parte más básica: una cuadrícula de 20×20 creada desde JavaScript. Después fui añadiendo las herramientas del editor para poder marcar inicio, meta, muros, costes y borrar celdas.

1. Crear el tablero 20×20 y generar las 400 celdas desde JavaScript.
2. Añadir los tipos de celda: inicio, meta, obstáculo, coste alto y borrado.
3. Evitar estados que no tenían sentido, por ejemplo que una celda fuera muro y meta a la vez.
4. Implementar los cuatro algoritmos obligatorios: BFS, DFS, UCS y A*.
5. Añadir la animación de celdas exploradas y la ruta final.
6. Mostrar las métricas: pasos, coste y número de celdas exploradas.
7. Crear cuatro escenarios fijos para poder repetir las pruebas.
8. Añadir el botón de comparar todos los algoritmos sobre el mismo mapa.
9. Añadir control de velocidad, pausa y la búsqueda bidireccional como mejora.
10. Subir el proyecto a GitHub y desplegarlo en Netlify.

2.1. Dificultades y correcciones

Una de las partes que más tuve que comprobar fue el tema de los costes. BFS y DFS buscan siguiendo el orden de las celdas, mientras que UCS y A* sí tienen en cuenta el coste acumulado. Por eso preparé un escenario con un camino corto pero caro, para comprobar que los resultados eran distintos.

También tuve que asegurarme de que todos los algoritmos trabajaran sobre exactamente el mismo mapa cuando se usa «Comparar todos». Si el escenario cambia entre una ejecución y otra, la comparación no tendría sentido.

Otra comprobación importante fue el caso sin solución. El programa no debe quedarse bloqueado: el algoritmo tiene que terminar y mostrar que no existe una ruta.

3. Arquitectura del proyecto

He separado el proyecto en archivos sencillos para que sea fácil localizar cada parte:

- index.html: contiene la estructura de la página, los paneles, botones, selector y zona del mapa.
- css/styles.css: contiene el aspecto visual de la aplicación, la cuadrícula, colores y adaptación a distintos tamaños.
- js/app.js: contiene la lógica del tablero, los escenarios, la animación y todos los algoritmos. 🖥️
- docs/: carpeta reservada para documentación, pruebas y capturas.
- assets/: carpeta para recursos adicionales si hacen falta.

3.1. Estructura de datos del mapa

Las posiciones del tablero se identifican con una clave formada por fila y columna, por ejemplo «10,2». El inicio se guarda en una variable, las metas y los muros en conjuntos (Set) y los costes en un Map. Con esto puedo consultar rápidamente si una celda es muro, meta o tiene un coste especial.

3.2. Flujo desde el clic hasta el resultado

- 1- El usuario selecciona una herramienta, por ejemplo «Obstáculo».
- 2- Hace clic sobre una celda.
- 3- JavaScript actualiza los datos del mapa y vuelve a dibujar esa celda.
- 4- Al pulsar «Ejecutar», se valida que exista inicio y al menos una meta.
- 5- Se ejecuta el algoritmo seleccionado y se guarda el orden de exploración y la ruta.
- 6- La interfaz anima primero las celdas exploradas y después la ruta final.
- 7- Finalmente se muestran pasos, coste y celdas exploradas.

4. Explicación de los algoritmos

4.1. BFS - búsqueda en anchura

BFS utiliza una cola. Va visitando primero las posiciones que están a menos pasos del inicio. Cuando todas las celdas tienen el mismo coste, encuentra una ruta con el menor número de pasos. Su punto débil en este proyecto es que no tiene en cuenta que una celda pueda costar más.

- Estructura principal: cola.
- Criterio: explorar por niveles, desde lo más cercano al inicio.
- Visitados: se marcan al añadirlos a la cola para no repetirlos.
- Ruta: se reconstruye guardando de qué celda viene cada posición.

4.2. DFS - búsqueda en profundidad

DFS utiliza una pila. En vez de explorar todo lo cercano primero, avanza todo lo posible por una dirección y después retrocede si se queda sin salida. Puede encontrar una ruta rápidamente, pero no garantiza ni el menor número de pasos ni el menor coste.

- Estructura principal: pila.
- Criterio: profundizar por una rama antes de probar otras.
- Visitados: impiden volver continuamente a las mismas celdas.
- Límite: el resultado depende bastante del orden en el que se añaden los vecinos.

4.3. UCS - búsqueda de coste uniforme

UCS elige siempre la opción con menor coste acumulado. Para eso usa una frontera ordenada por prioridad. Si todos los costes son no negativos, el primer camino que llega a la meta con el criterio de UCS tiene coste óptimo.

- Estructura principal: cola de prioridad (en el proyecto se ordena la frontera por coste).
- Criterio: menor coste acumulado desde el inicio.
- Con pesos: sí tiene en cuenta el valor de cada celda.
- Ventaja: encuentra el camino de menor coste aunque tenga más pasos.

4.4. A*

A* combina el coste real acumulado con una estimación de lo que falta hasta la meta. En este proyecto la heurística utilizada es la distancia Manhattan, porque solo se permiten movimientos arriba, abajo, izquierda y derecha.

$$f(n) = g(n) + h(n)$$

- $g(n)$: coste real acumulado desde el inicio.
- $h(n)$: distancia Manhattan hasta la meta más cercana.
- Criterio: se expande la posición con menor $f(n)$.
- En las pruebas con pesos, A* obtiene el mismo coste óptimo que UCS y normalmente explora menos celdas.

4.5. Búsqueda bidireccional (mejora)

Como mejora añadí una búsqueda bidireccional. La idea es hacer una búsqueda desde el inicio y otra desde la meta al mismo tiempo. Cuando las dos zonas exploradas se encuentran, se unen las dos partes de la ruta. En mapas grandes puede reducir bastante el número de celdas exploradas.

En mi implementación se basa en BFS, por lo que no usa los costes para decidir qué frente expandir. Por eso es útil como comparación de exploración, pero no sustituye a UCS o A* cuando hay pesos.

5. Cuatro laberintos de desafío

5.1. Escenario 1 - Menos pasos

Este mapa tiene caminos de distinta longitud y no tiene pesos. Sirve para comprobar que BFS encuentra una ruta mínima en número de pasos, mientras que DFS puede tomar un recorrido más largo.

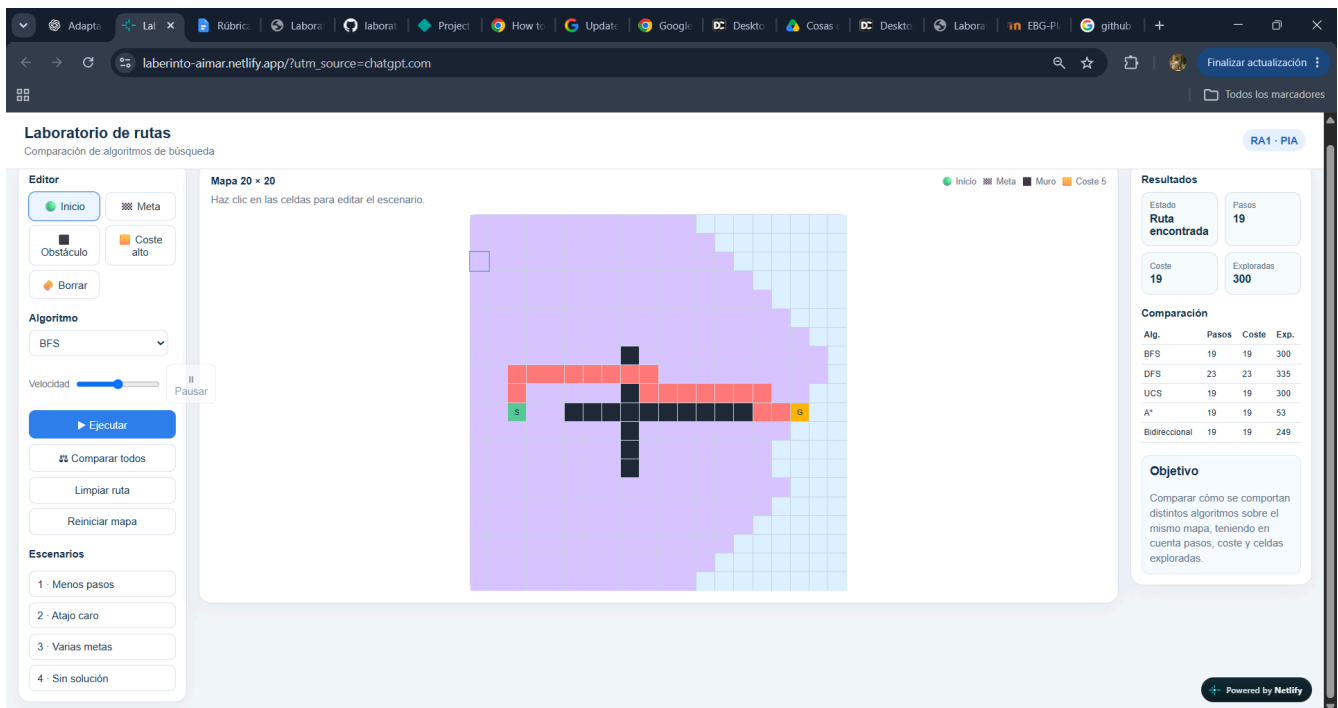


Figura 1. Escenario «Menos pasos» ejecutado con BFS y tabla de comparación.

5.2. Escenario 2 - Atajo caro

En este caso existe un camino directo, pero varias celdas de ese camino tienen coste 8. BFS elige el recorrido de 15 pasos y termina con coste 85. En cambio, UCS y A* prefieren dar un rodeo: hacen 17 pasos, pero el coste total baja a 17.

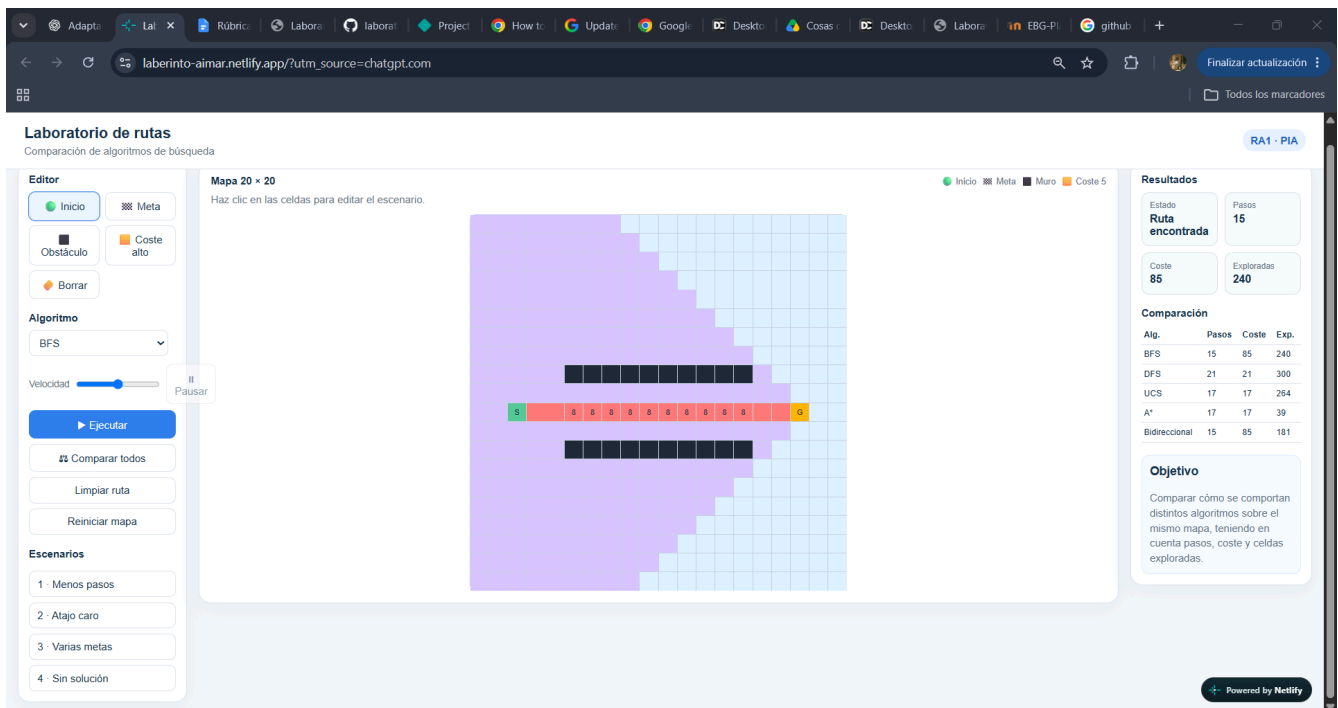


Figura 2. Escenario «Atajo caro»: menos pasos no significa menor coste.

5.3. Escenario 3 - Varias metas

Hay dos metas. La meta más cercana tiene una penalización de coste, mientras que la otra está más lejos pero sale más barata. BFS llega a la cercana en 10 pasos con coste 24. UCS y A* eligen la meta

lejana y consiguen coste 20.

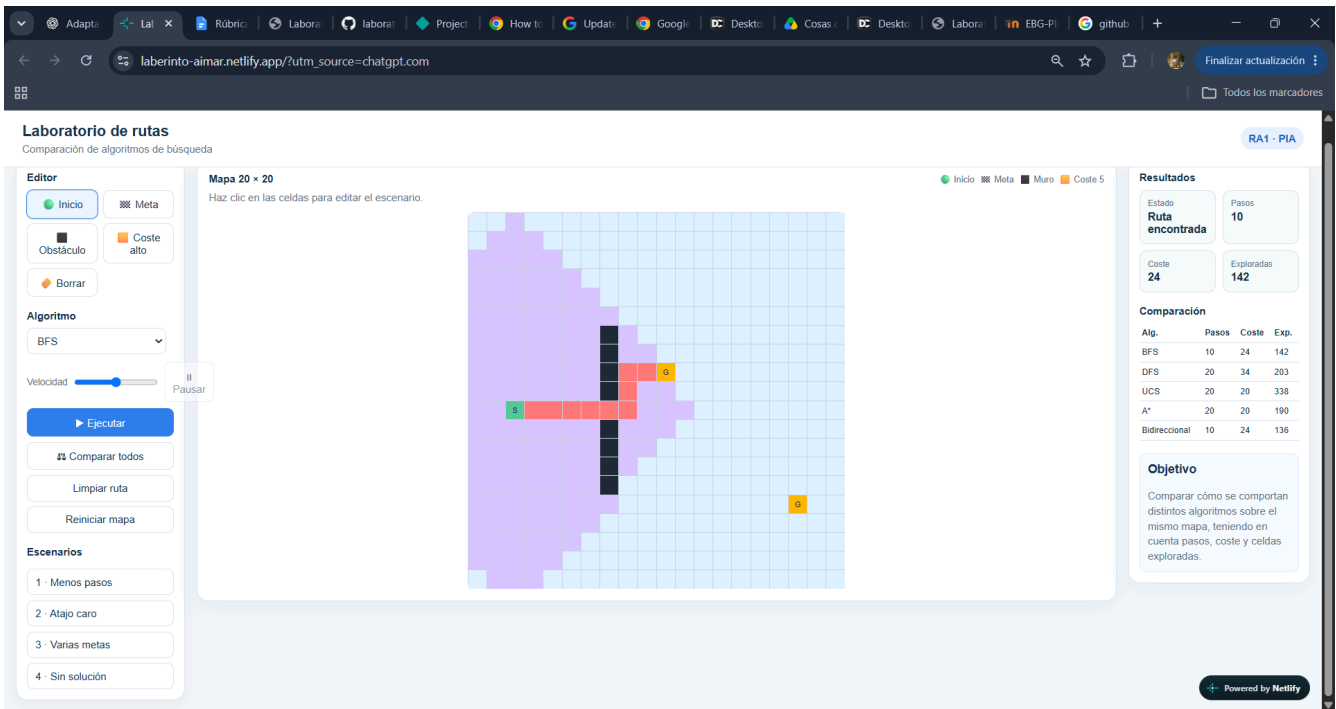


Figura 3. Escenario «Varias metas»: diferencia entre distancia y coste.

5.4. Escenario 4 - Sin solución

Una pared completa separa el inicio de la meta. El objetivo es comprobar que los algoritmos terminan correctamente y muestran «Sin solución», en vez de quedarse ejecutándose sin terminar.

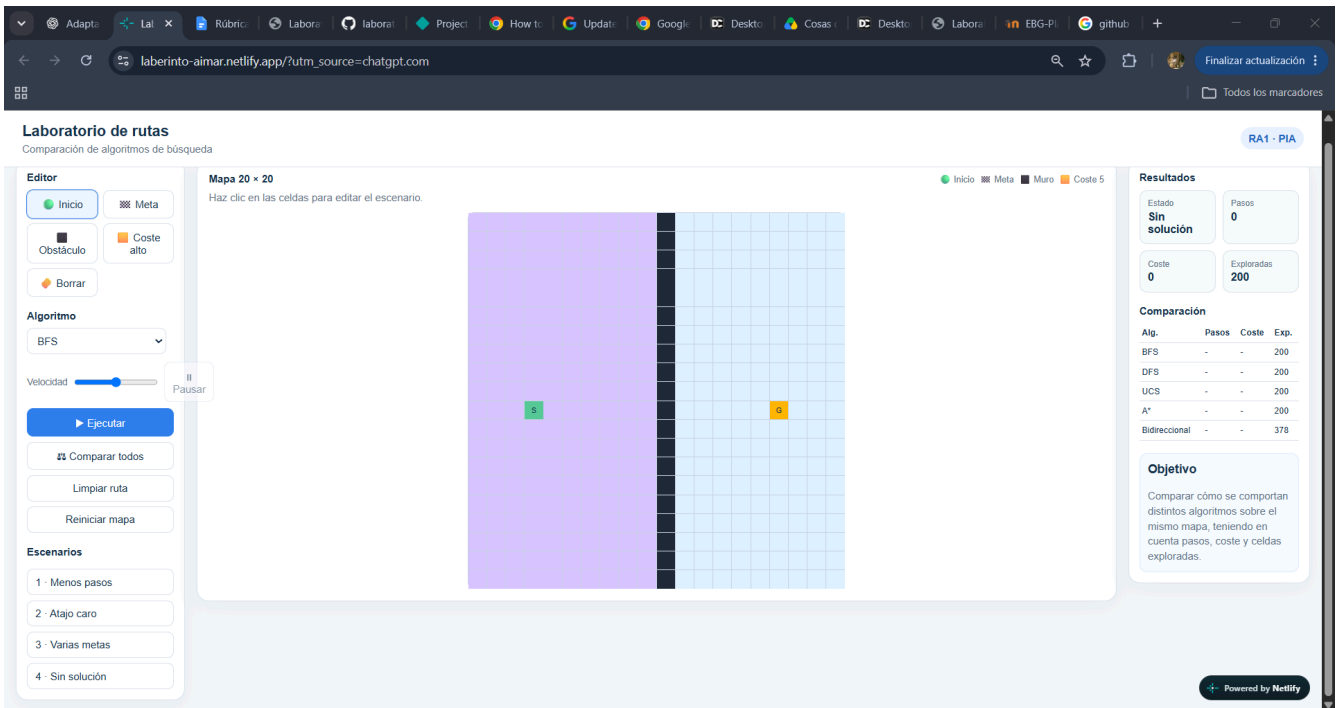


Figura 4. Escenario «Sin solución».

6. Comparación de resultados

Los datos siguientes son los que aparecen en la aplicación al pulsar «Comparar todos». En los escenarios con pesos se puede ver claramente la diferencia entre buscar menos pasos y buscar menor coste.

Escenario	Algoritmo	Pasos	Coste	Exploradas	Conclusión
1 Menos pasos	BFS	19	19	300	Ruta mínima en pasos
1 Menos pasos	DFS	23	23	335	Puede hacer más recorrido
1 Menos pasos	UCS	19	19	300	Mismo coste que BFS sin pesos
1 Menos pasos	A*	19	19	53	Mismo resultado explorando menos
2 Atajo caro	BFS	15	85	240	Menos pasos, pero mucho más caro
2 Atajo caro	DFS	21	21	300	Ruta válida no óptima en pasos
2 Atajo caro	UCS	17	17	264	Coste óptimo
2 Atajo caro	A*	17	17	39	Coste óptimo con menos exploración
3 Varias metas	BFS	10	24	142	Va a la meta cercana
3 Varias metas	DFS	20	34	203	Ruta dependiente del orden
3 Varias metas	UCS	20	20	338	Elige la meta más barata
3 Varias metas	A*	20	20	190	Mismo coste óptimo que UCS
4 Sin solución	BFS	-	-	200	Termina sin ruta
4 Sin solución	DFS	-	-	200	Termina sin ruta
4 Sin solución	UCS	-	-	200	Termina sin ruta
4 Sin solución	A*	-	-	200	Termina sin ruta

La prueba más clara es el escenario 2: BFS consigue 15 pasos, pero su coste es 85. UCS y A* hacen dos pasos más y bajan el coste a 17. En los escenarios 2 y 3, UCS y A* coinciden en el coste óptimo, que era una de las comprobaciones importantes.

7. Casos de prueba obligatorios

Prueba	Entrada	Esperado	Obtenido
Ruta simple	Ejecutar el escenario 1.	Se encuentra una ruta y se muestran métricas.	Correcto: BFS obtiene 19 pasos y coste 19.
Ausencia de inicio	Eliminar el inicio y pulsar Ejecutar.	No se ejecuta el algoritmo.	La interfaz muestra «Falta inicio».
Ausencia de meta	Eliminar todas las metas.	No se ejecuta el algoritmo.	La interfaz muestra «Falta meta».
Varias metas	Cargar escenario 3.	Los algoritmos pueden llegar a metas diferentes.	BFS prioriza la cercana; UCS y A* la de menor coste.
Pesos	Cargar escenario 2.	El coste influye en UCS y A*.	BFS: coste 85; UCS/A*: coste 17.
Obstáculos	Usar escenarios 1, 2 y 4.	Ninguna ruta atraviesa muros.	Correcto.
Mapa sin solución	Cargar escenario 4.	Los algoritmos terminan sin ruta.	Correcto, se muestra «Sin solución».
Editar después de ejecutar	Ejecutar y volver a editar el mapa.	Se puede seguir modificando.	Correcto, se limpia la visualización anterior.

8. Lenguaje de marcado, estilos y programación

8.1. HTML

HTML es el lenguaje de marcado. Lo utilizo para definir qué elementos existen en la página: cabecera, botones, paneles, selector de algoritmo, zona del mapa y resultados. HTML organiza el contenido, pero no ejecuta los algoritmos.

8.2. CSS

CSS se encarga de la presentación. Define tamaños, separación, colores y la apariencia de cada tipo de celda. Por ejemplo, los muros se muestran oscuros, el inicio verde, las metas amarillas, las celdas exploradas en morado y la ruta final en rojo.

8.3. JavaScript

JavaScript es el lenguaje de programación del proyecto. En él están las variables del mapa, los eventos de los botones, la lógica de edición, BFS, DFS, UCS, A*, la búsqueda bidireccional, la reconstrucción de la ruta y las animaciones.

Elegí JavaScript porque permite hacer toda la práctica directamente en el navegador y, para este proyecto, no hace falta instalar librerías ni usar un framework. Eso también facilita publicarlo en Netlify.

9. Mejoras implementadas

- Control de velocidad para ver la exploración más lenta o más rápida.
- Botón de pausa y reanudación durante la animación.
- Comparación automática de todos los algoritmos sobre el mismo mapa.
- Búsqueda bidireccional como algoritmo adicional.
- Interfaz adaptable con panel de edición, mapa y panel de resultados.

La mejora principal relacionada con algoritmia es la búsqueda bidireccional. No sustituye a los algoritmos obligatorios, sino que sirve para comparar otra forma de reducir la exploración cuando se conoce el origen y el destino.

10. Uso de asistentes de IA

Durante el trabajo he utilizado ChatGPT como apoyo. Lo he usado sobre todo para resolver dudas, proponer una forma de organizar el código, revisar errores y ayudarme a entender las diferencias entre los algoritmos. No he usado la IA solo para copiar y entregar: después de cada cambio he probado la aplicación y he comprobado los resultados en los escenarios.

Ejemplos de consultas que hice durante el proyecto:

- Cómo implementar y explicar BFS, DFS, UCS y A* en una cuadrícula.
- Cómo hacer que los obstáculos y los costes se puedan editar.
- Cómo comparar los cuatro algoritmos usando el mismo escenario.
- Cómo mostrar la exploración con animación, velocidad y pausa.
- Cómo organizar el proyecto para subirlo a GitHub y desplegarlo en Netlify.
- También fui cambiando cosas de las propuestas: el diseño de la interfaz, los escenarios, los controles y la forma de mostrar los resultados. Para verificarlo ejecuté los cuatro mapas y comparé pasos, coste y celdas exploradas.

Esta parte es importante porque, aunque he utilizado una herramienta de IA, necesito entender qué hace el código para poder explicarlo y modificarlo.

11. GitHub y Netlify

Repositorio de GitHub: <https://github.com/aimardf/laboratorio-rutas-ra1>

Aplicación publicada en Netlify: <https://laberinto-aimar.netlify.app>

El repositorio contiene el código fuente organizado y el README. Netlify está conectado al proyecto para que la aplicación se pueda abrir desde un enlace sin instalar nada.

12. Limitaciones

- El tablero no se guarda al cerrar o recargar la página.
- La búsqueda bidireccional está basada en BFS y no usa los pesos para elegir la frontera.
- A* utiliza distancia Manhattan, por lo que está pensado para movimiento ortogonal sin diagonales.
- La prioridad del proyecto ha sido entender los algoritmos y poder compararlos, no crear un sistema de mapas real.

13. Conclusión

Con este proyecto he podido ver mejor que «la ruta más corta» puede significar cosas distintas. BFS es útil cuando todas las celdas cuestan lo mismo y queremos pocos pasos. UCS busca el menor coste real y A* consigue el mismo objetivo usando una heurística para orientar la búsqueda. DFS es fácil de entender y puede encontrar una solución, pero no garantiza que sea la mejor.

Los escenarios me han servido para comprobarlo de forma visual. El caso del atajo caro es el más claro: BFS hace menos pasos, pero UCS y A* encuentran un camino mucho más barato. También he comprobado que el programa responde correctamente cuando hay varias metas o cuando no existe ninguna ruta